



Implement information extraction solutions

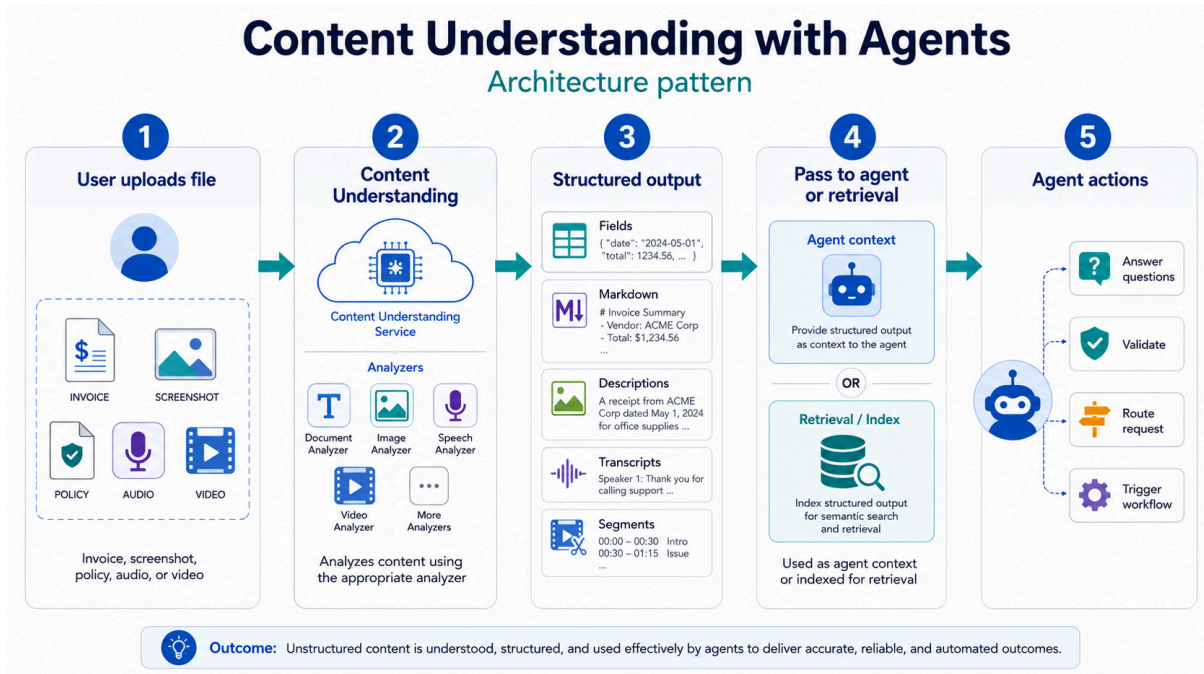
Lab - Azure Content Understanding - Document Analysis

What is Azure Content Understanding - <https://learn.microsoft.com/en-us/azure/ai-services/content-understanding/overview>

Analyzers in Azure Content Understanding

<https://learn.microsoft.com/en-us/azure/ai-services/content-understanding/concepts/analyzer-reference>

Content Understanding with Agents





In this chapter, we are looking at how Azure Content Understanding can work together with agents to process real business content and turn it into useful actions.

The process starts when a user uploads a file. This file could be an invoice, a screenshot, a policy document, an audio recording, or even a video. In many business applications, this kind of information is not already available in a clean structured format. It may be locked inside documents, images, recordings, or screenshots.

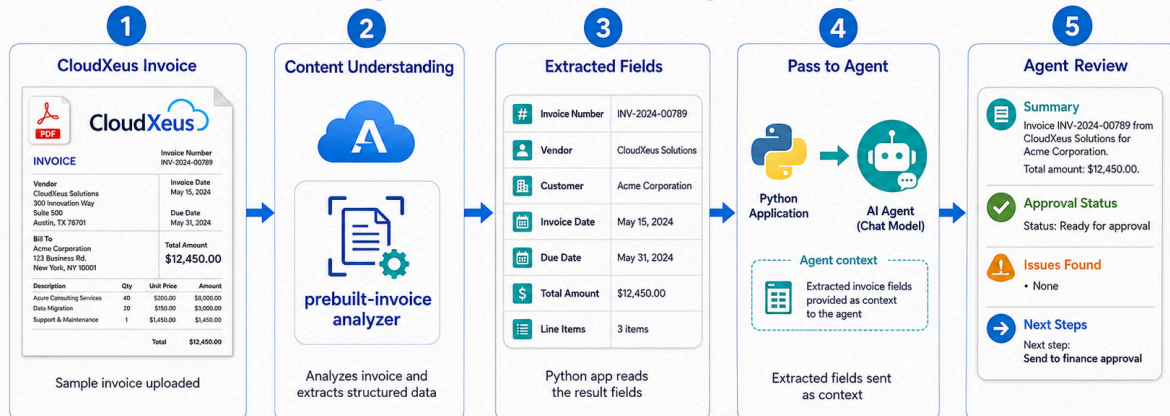
This is where Content Understanding comes in. The service uses the right type of analyzer based on the content. For example, a document analyzer can process forms and PDFs, an image analyzer can understand screenshots or pictures, a speech analyzer can work with audio, and a video analyzer can extract information from video content.

Once the content is analyzed, the output can be converted into structured information. This could include fields, markdown summaries, descriptions, transcripts, or segments. The important point is that the raw file is no longer just an unstructured upload. It becomes information that an application or an agent can work with. After that, the extracted information can be passed to an agent directly as context. Alternatively, it can be indexed into a retrieval system, so that the agent can search across it later. This is useful when the content needs to support question answering, validation, routing, or workflow automation.

The final step is where the agent takes action. The agent can answer a user's question, validate the extracted information, route a request to the correct team, or trigger another workflow. So the pattern is not just about extracting data. It is about using that extracted data to support smarter business decisions and automated outcomes.

Invoice Review Agent

Using Azure Content Understanding and an AI agent



Outcome: Content Understanding extracts the invoice data, and the agent reasons over it to produce a business-friendly recommendation.

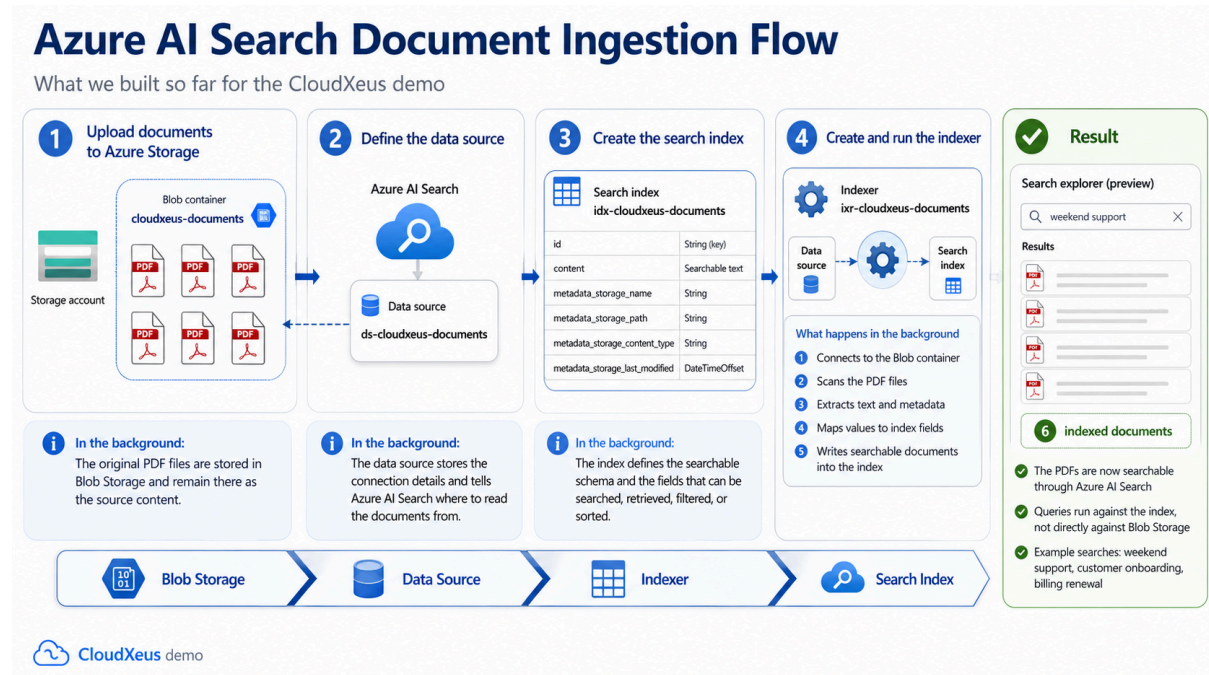


The above image shows this idea with a practical invoice review scenario. A sample CloudXeus invoice is uploaded into the application. Azure Content Understanding uses the prebuilt invoice analyzer to extract structured invoice data, such as the invoice number, vendor, customer, invoice date, due date, total amount, and line items.

A Python application then reads these extracted fields and passes them as context to an AI agent. This is important because the agent does not need to guess what is in the invoice. It receives the actual extracted values and can reason over them in a more reliable way. The agent can then produce a business-friendly review. For example, it can summarize the invoice, identify whether it is ready for approval, check if any issues were found, and recommend the next step. In the image, the agent confirms that the invoice is ready for approval and suggests sending it for finance approval.

The key takeaway is that Content Understanding handles the extraction, while the agent handles the reasoning and business response. Together, they help convert uploaded files into structured information, and then into meaningful actions. This is especially valuable for scenarios like invoice review, claims processing, support ticket analysis, compliance checks, and document-based workflows.

Lab - Ingesting documents from Azure Blob Storage





We are looking at how to ingest documents from Azure Blob Storage into Azure AI Search so that the content becomes searchable and usable in a retrieval-based application.

The flow starts with the original PDF documents being uploaded to a Blob Storage container. In this example, the documents are stored in a container named cloudeus-documents. These files remain in Blob Storage as the source content. Azure AI Search does not replace Blob Storage. Instead, it reads the files from storage and builds a searchable index from them.

Next, we define a data source in Azure AI Search. The data source stores the connection information that tells Azure AI Search where the documents are located. In this case, the data source points to the Blob Storage container that contains the PDF files. This step is important because the search service needs a trusted source from which it can read the documents during indexing.

After the data source is created, we create the search index. The index defines the searchable structure of the data. It includes fields such as the document ID, content, storage path, file name, content type, and last modified date. Some fields can be searchable, while others can be used for filtering, sorting, or retrieving metadata. This gives us control over how the documents can be queried later.

The next step is to create and run the indexer. The indexer connects to the data source, scans the PDF files, extracts the text and metadata, maps the extracted values to the fields in the index, and writes searchable documents into the search index. This is the background process that turns files in Blob Storage into searchable records in Azure AI Search.

Once the indexer completes, the result is a populated search index. In the example shown, the search explorer is used to run a query such as "weekend support." The results come from the Azure AI Search index, not directly from Blob Storage. This means the documents have now been prepared for fast search and retrieval.